

Effectiveness of a Convolution Neural Network when Categorising Synthetically Generated Gaussian Blurs

Introduction

This project aims to evaluate the effectiveness of a Convolution Neural Network for image classification, when the inputted image are two types of synthetically generated Gaussian Blurs.

During this experiment, I will run a series of Parametric Experiments, covering model breadth and depth in addition to Hyper-Parameter Tuning to produce the greatest set of results.

I intend to employ Sagemaker's Model Training functionality, rather than the EC2 setup used in previous projects. Experiments will be deployed to a Sagemaker gl4 instance via the Python Sagemaker SDK.

Introduction to The Dataset

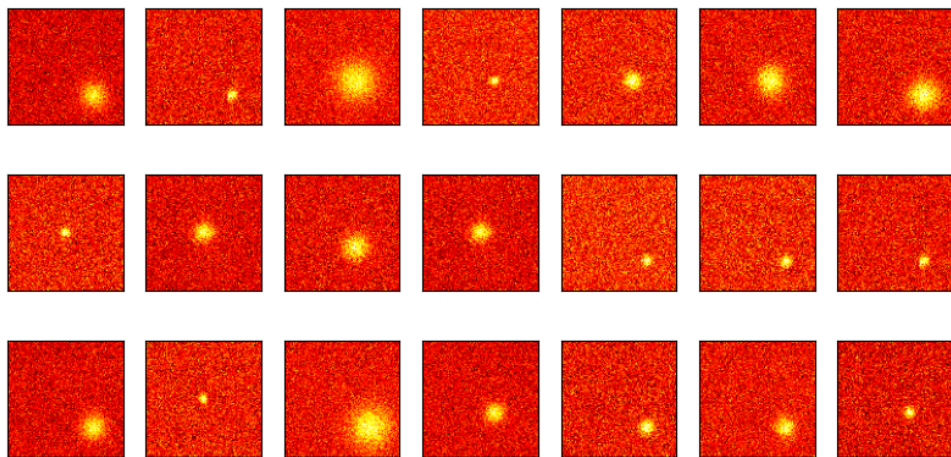


Figure 1 - Exemplar Randomly Generated Gaussian Blurs

Isaac Nesbeth

The data used as part of this experiment was synthetically generated and was produced following the standard Gaussian equation.

The centre coordinates of the Gaussian were randomly generated and were combined alongside a randomly generated, minute centre offset.

Two classes of Gaussian were generated based on the width of the Gaussian. Figure 1 depicts these classes.

The widths were randomly selected from two previously defined sets of class widths.

Class 0 (Large widths):

`array([1. , 1.5, 2. , 2.5, 3. ])`

Class 1 (Small widths):

`array([3.5, 4. , 4.5, 5. , 5.5])`

Parametric Experiment 1

This experiment evaluated how the size of the Feed Forward Layer affected overall CNN performance.

The experiment parameters were as follows:

5 layers:

`[100, 75, 50, 25, 1]`

10 layers:

`[100, 89, 78, 67, 56, 45, 34, 23, 12, 1]`

15 layers:

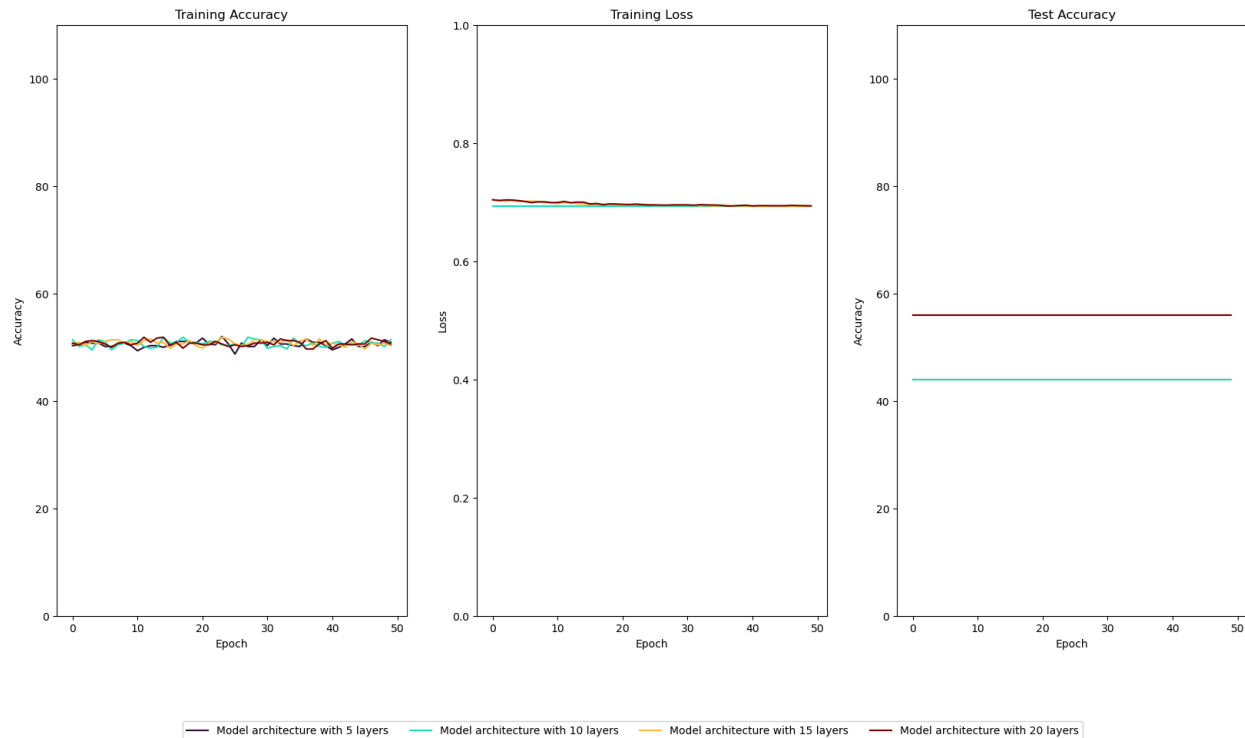
`[100, 92, 85, 78, 71, 64, 57, 50, 43, 36, 29, 22, 15, 8, 1]`

20 layers:

`[100, 94, 89, 84, 79, 73, 68, 63, 58, 53, 47, 42, 37, 32, 27, 21, 16, 11, 6, 1]`

Each item represents the tensor feature input and output sizes per layer and subsequent layer. We can observe that the increase in layer size reduced the tensor feature input/ output size.

Experiment Results:



The graphs above depict results for Training Accuracy, Training Loss and Test Accuracy. We can see that there were no major differences in Training Accuracy and Loss across the four experimental architectures. This demonstrates that the overall architecture was unable to learn the relationships between the various features, but indicates that the issue stemmed from the CNN architecture rather than FFN architecture as the trend was noticed across all architectures.

An interesting observation is the Test Accuracy variation between the model with 20 layers and 10 layers. This is likely to be a result of a combination of model breadth and depth. In the 10 layer model, there was less depth however each layer had significantly fewer hidden nodes. On the other hand, in the 20 layer model, there was more depth however each layer had more hidden nodes (this is a result of the number of nodes per layer systematically decreasing from 100 to 1). The reduction in hidden nodes per layer in the 10 layer model was more substantial than the 20 layer model, meaning that a greater number of data relationships was lost since the model had to aggressively compress data in the 10 layer model whilst gradually compressing relationships in the 120 layer model.

Similarly, the 20 layer model had greater capacity for learning feature representations than the 10 layer model, contributing to its higher Test Performance.

Parametric Experiment 2

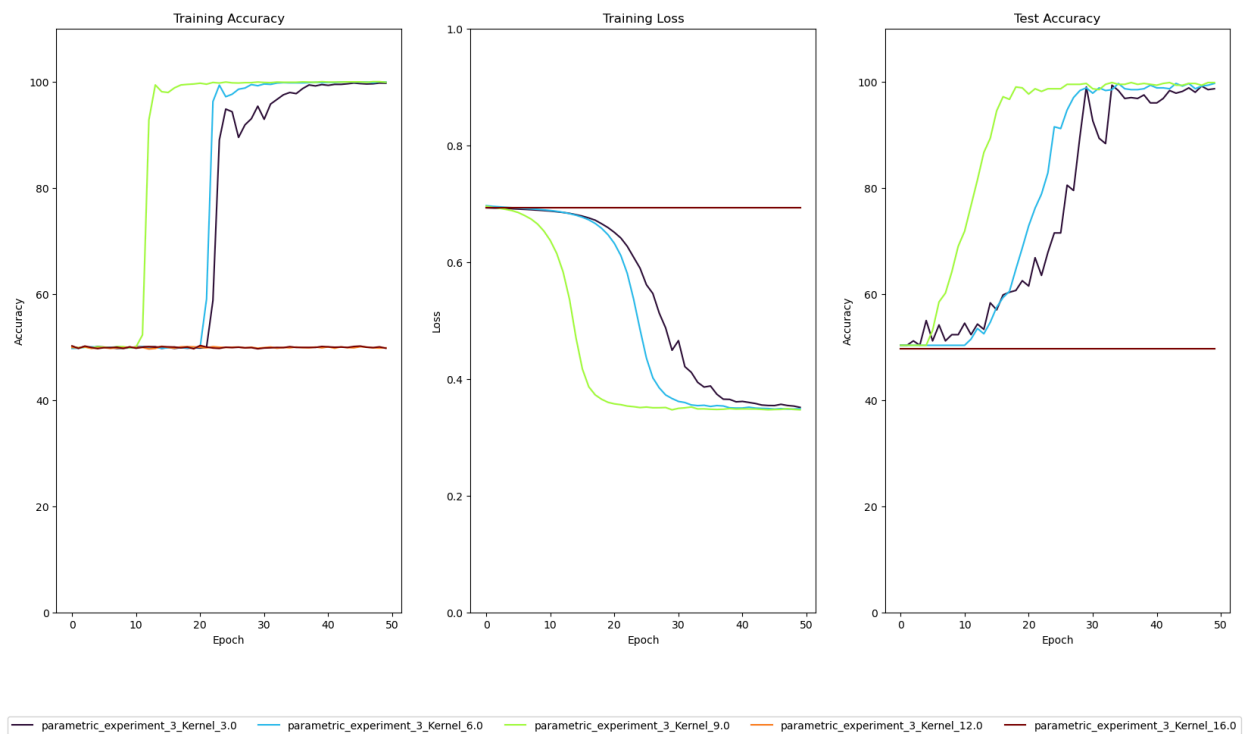
In this experiment, we assessed how kernel size impacted model performance. This experiment was deployed to AWS Sagemaker Model Trainer in line with previous experiments.

Kernel sizes were systematically generated using Python numpy's() linspace method, with a maximum kernel size of 16 x 16.

The parameters used in the experiment were:

[3, 6, 10, 13, 16]

Experiment Results



We can see that the experiment with the kernel sizes set to 16x16 and 12x12 performed very poorly compared with other kernel size experiments. All other kernel sizes Throughout the 50 epochs, the training loss remained consistent and both training accuracy and test accuracy stagnated at approximately 50%. These results demonstrate that the 16x16 and 12x12 kernel size was not able to learn the relationships between pixel features. As each synthetic image was of size 91x91, both the 16x16 and 12x12 kernel compressed and reduced pixel relationships (as a result of conducting the dot product on the 16x16 image area and kernel) into a single value which as a result summarise and deprioritised key features.

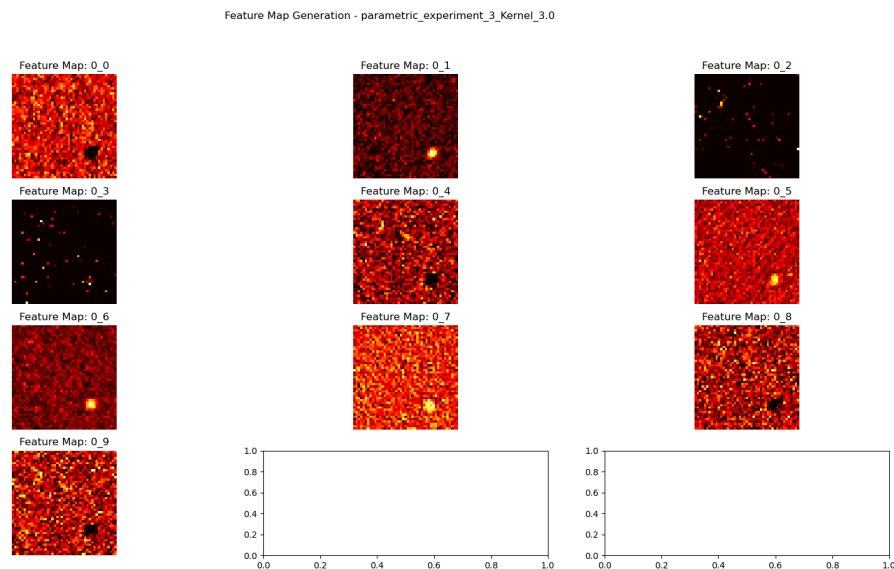
The order of kernel sizes where the overall model architecture achieved 100% is another interesting observation. The order of experiments which reached 100% from greatest to least amount of epochs.

1. Parametric Experiment 1 - Kernel Size 3x3
2. Parametric Experiment 2 - Kernel Size 6x6
3. Parametric Experiment 3 - Kernel Size 9x9
4. Parametric Experiment 4 and 5 - Kernel Size 12x12 and 16x16

In terms of the fewest number of training epochs, the optimum kernel size was 9x9. This size enabled the model to learn smaller but significant pixel relationships and balanced an appropriate receptive field with reduced data loss.

On the other hand, the 3x3 kernel size achieved 100% after approximately 35 training epochs. Despite the smaller kernel size enabling smaller relationships to be learnt, combined with the lower number of CNN layers, the 3x3 kernel resulted in an overall smaller receptive field than the 9x9 kernel. Therefore the 3x3 kernel required more epochs than the 9x9 kernel to build the same receptive picture and learn complex and overall pixel relationships. If the CNN had additional layers then this might have mitigated the loss in receptive field.

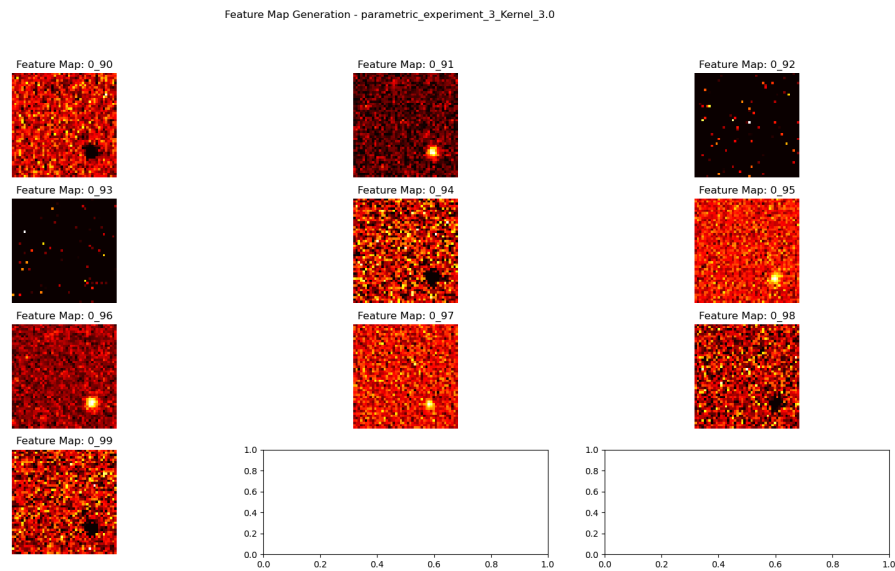
In this experiment, we also examined the various experiment feature maps per layer. Feature maps refer to the output of a kernel and image/ previous layer.



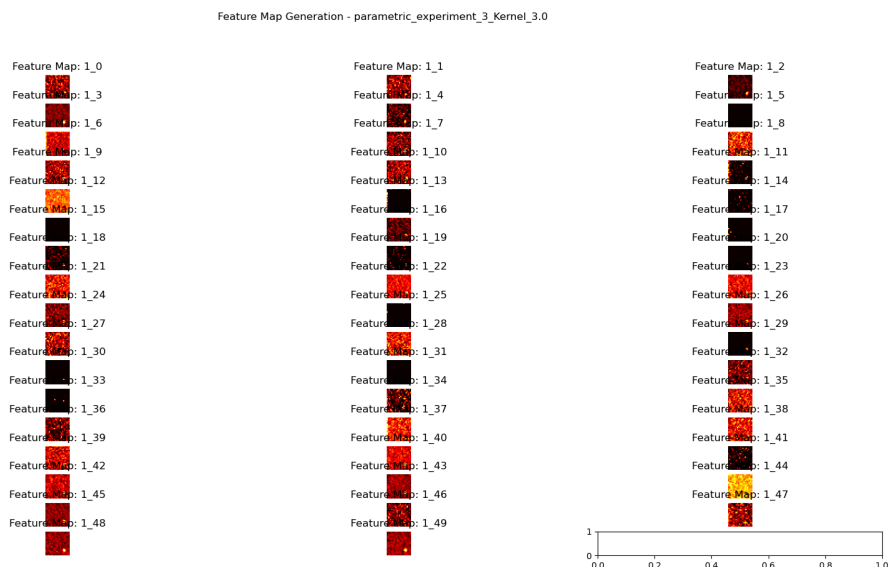
If we look at the Feature Map when the kernel size was equal to 3x3 above, we can see that nine feature maps were produced, and therefore nine kernels were implemented. The feature map above represents the output of the first layer of the model. We can see that the model developed kernels where it was able to separate the Gaussian from the background noise, apart

from in Feature Maps 2 and 3 where it produced random noise, because there are small circles present where Gaussians were located.

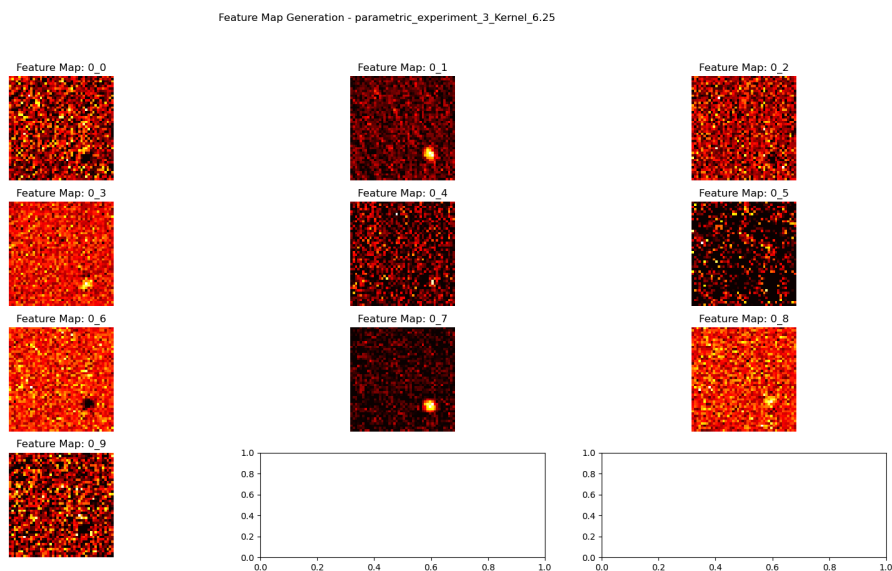
If we look at another Gaussian Feature Map for Layer 1 below, we can see the same pattern - the model produced kernels where it could separate the Gaussian from background noise.



However, when looking at Layer 2 onwards, we can begin to identify the common disadvantage of CNN's and Deep Learning generally - the lack of explainability. As more and more feature maps are produced, it becomes trickier to discern and understand the model's inferencing/ decision intentions.

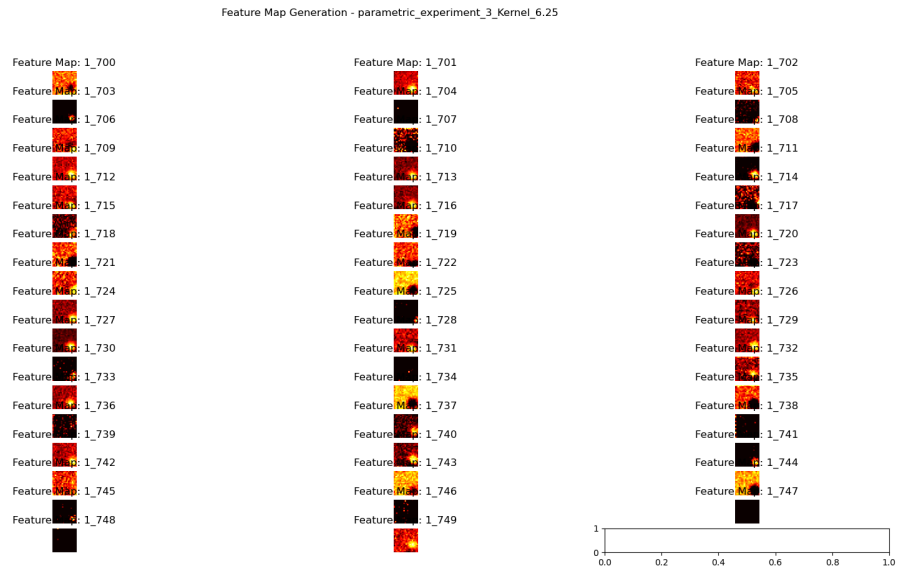


If we now explore Feature Maps when the kernel size was equal to 6x6, we can see that the model developed Gaussian awareness at further layers compared to the immediate understanding we see in the 3x3 kernel size.



Whilst some 6x6 kernels were able to develop situational awareness for the position of the Gaussian, it is clear that the output is heavily distorted by noise and it is not clear that a Gaussian is therefore present. For example, if we refer to Feature Maps 1 and 7, it is quite clear that a Gaussian is present. In Feature Maps 3 and 6, it's reasonably clear but in Feature Maps 0 and 5 only noise is visible.

However, when looking at Layer 1 of the 6x6 Kernel Experiment, we can see the model was able to learn Gaussian features and apply this learned information to the kernels.



In almost all feature maps produced, we can see that kernels produced feature maps with clear Gaussians present and minimal noise distortion. This suggests that the model initially extracted noise from the image in order to apply a secondary transformation which extracted clear gaussian features.

Conclusion

This project demonstrated that CNN classification performance on synthetic Gaussian blurs is heavily dictated by receptive field size and feature compression dynamics.

- **Kernel Size & Receptive Field:** A 9×9 kernel achieved optimal performance (100% accuracy in the fewest epochs) by capturing sufficient spatial context without destroying pixel relationships. Smaller kernels (3×3) required significantly more epochs due to a constrained receptive field, while larger kernels (12×12 and 16×16) failed entirely (~50% accuracy) by over-compressing spatial features into uninformative averages.
- **FFN Depth & Compression Rate:** Gradual feature reduction across layers (as seen in the 20-layer model) better preserved critical representations compared to aggressive compression in shallower, narrower architectures.
- **Feature Map Interpretability:** Early CNN layers successfully isolated Gaussian centroids from background noise, while deeper layers transitioned into abstract, unexplainable feature representations.

Ultimately, tuning kernel sizes to match the physical scale of target features is more critical for spatial signal extraction than simply increasing model depth.